

# **SUMMER INTERNSHIP PROJECT REPORT**

## **AUTOMATED PHI/PII REDACTION PIPELINE FOR LLMs**

*Submitted in partial fulfillment of the requirements for the degree of*  
**Bachelor of Technology in Computer Science and Engineering**

*Submitted by:*  
**Patel Mantra Kalpeshkumar (24DCS076)**

*Under the guidance of:*  
**Dr. Khushi Patel (Internal Guide)**  
**Chandan Mishra (External Mentor, Infotact Solutions)**

**DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH  
(DEPSTAR)**

**Charotar University of Science and Technology (CHARUSAT)**

July, 2026

**DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND  
RESEARCH  
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**CERTIFICATE**

This is to certify that the internship report titled "Automated PHI/PII Redaction Pipeline for LLMs" is a bonafide work carried out by Patel Mantra Kalpeshkumar (24DCS076) under my supervision and guidance during the summer internship period. The work has been carried out satisfactorily and meets the academic requirements for the degree of Bachelor of Technology in Computer Science and Engineering.

---

**Dr. Khushi Patel**  
Internal Project Guide

---

**Head of Department**  
Department of CSE

# **INFOTACT SOLUTIONS**

**Bengaluru, Karnataka**

## **TO WHOMSOEVER IT MAY CONCERN**

This is to certify that Patel Mantra Kalpeshkumar (24DCS076), a student of DEPSTAR, CHARUSAT, has successfully completed a 2-month summer internship at Infotact Solutions from May 2026 to July 2026. During this period, he worked in the role of Security Admin Associate L1 on the project "Automated PHI/PII Redaction Pipeline for LLMs". His work primarily involved designing regular expression pipelines, backend gateway router API endpoints, and rotated compliance audit logging controllers. His conduct and performance during the internship were highly commendable.

**Chandan Mishra**  
**External Internship Mentor**  
**Infotact Solutions**

## ACKNOWLEDGEMENT

First and foremost, I would like to express my sincere gratitude to Infotact Solutions for giving me the opportunity to work as a Security Admin Associate L1 during my 2-month summer internship. This project provided me with invaluable experience in integrating security and compliance guardrails directly into backend clinical software architectures.

I am highly indebted to my external mentor, Chandan Mishra, for his technical guidance, code reviews, and constant encouragement throughout the design and implementation of the token vault mechanisms. I am also extremely grateful to my project guide, Dr. Khushi Patel, and the faculty members of DEPSTAR, CHARUSAT, for providing structural direction and academic feedback.

Finally, I want to thank my teammates Rohit Suthar, Tirth Patel, and Jash Joshi. Our collaborative integration efforts—merging the frontend dashboard, the Presidio NLP engines, and the Redis token mappings—were essential in delivering a highly stable, functional prototype of a privacy-preserving proxy pipeline.

**Patel Mantra Kalpeshkumar**  
**Enrollment: 24DCS076**

## ABSTRACT

Integrating Large Language Models (LLMs) into healthcare systems exposes sensitive patient records to compliance and privacy leakage risks. Under HIPAA regulations, clinical data containing Protected Health Information (PHI) or Personally Identifiable Information (PII) must be redacted prior to being routed outside secure organizational boundaries. This project implements a secure, de-identification proxy acting as a compliance firewall.

The pipeline utilizes a hybrid detection architecture. A stateless rules-based Regex scanner detects 13 distinct types of structured identifiers (such as Aadhaar cards, SSNs, medical record numbers, ZIP codes, and dates), while Microsoft Presidio and spaCy handle unstructured entities like patient names and locations. Overlapping matches are resolved using a length-priority greedy matrix to avoid nested double-redactions. Anonymized token mappings (e.g. PERSON\_001) are Fernet-encrypted and stored in a Redis session vault with an auto-destruction 30-minute Time-to-Live (TTL) policy. Finally, original identities are dynamically restored in the safe LLM responses before being delivered back to clinicians.

To satisfy prototype pipeline design goals, the system integrates sliding-window IP rate-limiting, size-based rotating audit logging (logging metadata only to prevent PHI leaks), and a stateless pre-check route executing in  $\sim 1.29\text{ms}$  to warn users of sensitive data exposure risks. Programmatic validation via a 65-assert Pytest suite confirms pipeline durability.

# TABLE OF CONTENTS

|                                                                 |    |
|-----------------------------------------------------------------|----|
| Acknowledgement .....                                           | iv |
| Abstract .....                                                  | v  |
| Description of Company / Organization .....                     | ix |
| CHAPTER 1 — INTRODUCTION .....                                  | 1  |
| 1.1 BACKGROUND .....                                            | 1  |
| 1.2 DATA PRIVACY IN HEALTHCARE .....                            | 1  |
| 1.3 PERSONALLY IDENTIFIABLE INFORMATION (PII) .....             | 1  |
| 1.4 PROTECTED HEALTH INFORMATION<br>(PHI) .....                 | 2  |
| 1.5 PRIVACY RISKS IN LARGE LANGUAGE MODEL<br>APPLICATIONS ..... | 2  |
| 1.6 NEED FOR AUTOMATED PHI/PII<br>REDACTION .....               | 2  |
| 1.7 PROBLEM STATEMENT .....                                     | 2  |
| 1.8 OBJECTIVES OF THE PROJECT .....                             | 3  |
| 1.9 SCOPE OF THE PROJECT .....                                  | 3  |
| CHAPTER 2 — PROJECT DEFINITION .....                            | 4  |
| 2.1 PROJECT OVERVIEW .....                                      | 4  |
| 2.2 EXISTING SYSTEM .....                                       | 4  |
| 2.3 PROPOSED SYSTEM .....                                       | 4  |
| 2.4 TECHNOLOGY STACK .....                                      | 4  |
| 2.5 KEY FEATURES OF THE SYSTEM .....                            | 5  |
| CHAPTER 3 — SYSTEM REQUIREMENTS .....                           | 6  |
| 3.1 SOFTWARE REQUIREMENTS .....                                 | 6  |
| 3.2 HARDWARE REQUIREMENTS .....                                 | 6  |

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| 3.3 DEVELOPMENT AND VALIDATION TOOLS .....                                | 7  |
| CHAPTER 4 — SYSTEM DESIGN AND ARCHITECTURE .....                          | 8  |
| 4.1 SYSTEM ARCHITECTURE .....                                             | 8  |
| 4.2 SYSTEM WORKFLOW .....                                                 | 8  |
| 4.3 REDIS KEY-SPACE AND SESSION-SCOPED TOKEN MAPPING<br>MODEL .....       | 9  |
| 4.4 SECURITY ARCHITECTURE .....                                           | 10 |
| CHAPTER 5 — SYSTEM IMPLEMENTATION .....                                   | 12 |
| 5.1 FRONTEND IMPLEMENTATION .....                                         | 12 |
| 5.2 BACKEND IMPLEMENTATION .....                                          | 12 |
| 5.3 VAULT & TOKEN STORAGE<br>IMPLEMENTATION .....                         | 13 |
| 5.4 PRESIDIO/SPACY NLP PROCESSING AND CLINICAL TERM<br>PRESERVATION ..... | 14 |
| 5.5 IMPLEMENTATION CHALLENGES &<br>SOLUTIONS .....                        | 15 |
| CHAPTER 6 — WORKFLOW AND MAJOR<br>FUNCTIONALITY .....                     | 17 |
| 6.1 MAIN REQUEST LIFECYCLE .....                                          | 17 |
| 6.2 REGEX SCANNING & OVERLAP<br>RESOLUTION .....                          | 17 |
| 6.3 TOKEN GENERATION & DE-<br>IDENTIFICATION .....                        | 18 |
| 6.4 RESPONSE GENERATION AND IDENTITY<br>RESTORATION .....                 | 19 |
| 6.5 AUDIT TRAIL LOGGING .....                                             | 20 |



|                                                                       |    |
|-----------------------------------------------------------------------|----|
| CHAPTER 7 — TESTING AND VALIDATION .....                              |    |
| 21                                                                    |    |
| 7.1 TESTING OBJECTIVES .....                                          | 21 |
| 7.2 TESTING STRATEGY .....                                            | 21 |
| 7.3 UNIT TESTING .....                                                | 21 |
| 7.4 INTEGRATION TESTING .....                                         |    |
| 22                                                                    |    |
| 7.5 TEST CASES & RESULTS .....                                        | 22 |
| CHAPTER 8 — RESULTS AND OUTPUTS .....                                 |    |
| 23                                                                    |    |
| 8.1 APPLICATION STARTUP &<br>CONFIGURATION .....                      | 23 |
| 8.2 MAIN GATEWAY USER INTERFACE .....                                 |    |
| 23                                                                    |    |
| 8.3 INPUT NOTE DE-IDENTIFICATION<br>RUN .....                         | 23 |
| 8.4 RESPONSE GENERATION AND IDENTITY RESTORATION<br>RUN .....         | 24 |
| 8.5 AUDIT LOGS & COMPLIANCE<br>METRICS .....                          | 25 |
| CHAPTER 9 — SECURITY, PRIVACY, AND LIMITATIONS .....                  |    |
| 28                                                                    |    |
| 9.1 TECHNICAL LIMITATIONS .....                                       | 28 |
| 9.2 SYSTEM AND SCALABILITY<br>LIMITATIONS .....                       | 28 |
| 9.3 SECURITY LIMITATIONS .....                                        | 28 |
| CHAPTER 10 — FUTURE ENHANCEMENTS .....                                |    |
| 30                                                                    |    |
| 10.1 MODEL OPTIMIZATIONS .....                                        | 30 |
| 10.2 SECURITY, PRIVACY, AND COMPLIANCE-ORIENTED<br>ENHANCEMENTS ..... | 30 |
| CONCLUSION .....                                                      | 32 |

|                  |    |
|------------------|----|
| REFERENCES ..... | 33 |
|------------------|----|

## LIST OF FIGURES

|                                                           |    |
|-----------------------------------------------------------|----|
| Fig. 3.1 Local Development & Validation Flowchart .....   | 7  |
| Fig. 4.1 Overall System Architecture<br>Diagram .....     | 8  |
| Fig. 4.2 Hybrid Redaction Pipeline<br>Flowchart .....     | 8  |
| Fig. 4.3 End-to-End Request Lifecycle Sequence .....      | 9  |
| Fig. 4.4 Redis Key-Space Layout .....                     | 9  |
| Fig. 4.5 Trust Boundary Controls Map .....                | 10 |
| Fig. 5.1 Project Directory Tree .....                     | 12 |
| Fig. 5.2 Presidio + spaCy NLP Processing<br>Flow .....    | 15 |
| Fig. 6.1 Greedy Overlap Resolution Flow .....             | 18 |
| Fig. 6.2 Token Storage and Restoration<br>Flow .....      | 18 |
| Fig. 6.3 Audit Logging Rotated Flow .....                 | 20 |
| Fig. 6.4 Rate Limiting Sliding Window<br>Flow .....       | 20 |
| Fig. 8.1 Flask startup diagnostics printout .....         | 23 |
| Fig. 8.2 Frontend login viewport selection .....          | 23 |
| Fig. 8.3 EHR Proxy Dashboard viewport .....               | 23 |
| Fig. 8.4 Redacted and de-redacted text<br>viewports ..... | 23 |
| Fig. 8.5 Redis mapped token store key lists .....         | 24 |

|                                                             |    |
|-------------------------------------------------------------|----|
| Fig. 8.6 Rotated audit log diagnostic files .....           |    |
| 25                                                          |    |
| Fig. 8.7 Pytest local suite execution stdout .....          |    |
| 25                                                          |    |
| Fig. 8.8 Regex pattern validator outputs .....              | 25 |
| Fig. 8.9 Local HIPAA safe harbor evaluation<br>report ..... | 25 |
| Fig. 8.10 Accuracy scorer benchmarking<br>statistics .....  | 25 |

## LIST OF TABLES

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| Table 1.1 HIPAA Safe Harbor 18 Protected Identifiers Mapping Matrix ..... | 2  |
| Table 2.1 Technology Stack Frameworks and Verification Matrix .....       | 4  |
| Table 3.1 Software Dependencies Requirements .....                        | 6  |
| Table 3.2 Minimum and Recommended Hardware Specifications .....           | 6  |
| Table 5.1 Flask Gateway Proxy API Routing Directory .....                 | 12 |
| Table 5.2 Microsoft Presidio NLP Analyzer Configurations .....            | 14 |
| Table 6.1 Rules-based Regex priority overlap rankings .....               | 17 |
| Table 7.1 Pytest local test suite execution layout .....                  | 21 |
| Table 7.2 Pattern validator assertion test configurations .....           | 22 |
| Table 7.3 Test execution metrics summary table .....                      | 22 |

## **DESCRIPTION OF COMPANY / ORGANIZATION**

Infotact Solutions is a technology firm based in Bengaluru, Karnataka, providing consulting and custom software integration solutions. The student completed a 2-month online summer internship in the role of Security Admin Associate L1, under the external mentorship of Chandan Mishra (External Mentor) and internal academic guidance of Dr. Khushi Patel (Internal Guide). The project assigned was a team-based task titled "Automated PHI/PII Redaction Pipeline for LLMs" and focused on implementing de-identification algorithms and auditing wrappers.

Due to the remote online nature of the internship, detailed internal organizational charts, departmental blocks, or client account portfolios remain outside the primary scope of accessible information. The project focuses purely on technical contributions to the proxy repository provided by Infotact Solutions.

## CHAPTER 1 — INTRODUCTION

### 1.1 BACKGROUND

Modern healthcare ecosystems rely extensively on digitized medical record storage and integration frameworks. While digital integration speeds clinical decision support and improves administrative efficiency, it exposes highly sensitive patient information to cyber breaches and data leaks. As healthcare systems incorporate artificial intelligence (AI) and large language models (LLMs) to automatically summarize doctor notes or suggest next diagnostic checks, protecting sensitive patient details becomes a top priority. Raw patient notes must be de-identified before they are configured to be forwarded to cloud-based or third-party LLM providers.

### 1.2 DATA PRIVACY IN HEALTHCARE

Data privacy in healthcare is governed by strict regulatory rules such as the Health Insurance Portability and Accountability Act (HIPAA) in the United States and the Digital Personal Data Protection (DPDP) Act in India. Under HIPAA rules, Protected Health Information (PHI) includes any health status, provision of healthcare, or healthcare payment data that can be linked to a specific patient. Organizations that fail to de-identify data prior to external storage or API dispatch face severe financial penalties and legal liability.

### 1.3 PERSONALLY IDENTIFIABLE INFORMATION (PII)

Personally Identifiable Information (PII) represents any data that can be used to distinguish or trace an individual's identity, either alone or when combined with other public records. Examples include full names, national identifiers (Aadhaar cards, Social Security Numbers), email addresses, and phone contacts. In clinical settings, PII is frequently intertwined with medical observations, requiring careful extraction.

### 1.4 PROTECTED HEALTH INFORMATION (PHI)

Under the HIPAA Safe Harbor standard, 18 specific categories of identifiers are classified as protected. The system maps its regular expression scanners and NLP extraction rules directly to these categories to support HIPAA compliance alignment. These 18 identifiers include

patient names, dates, phone numbers, fax numbers, email addresses, Social Security Numbers, medical record numbers (MRNs), health plan beneficiary numbers, account numbers, license numbers, vehicle IDs, device serial numbers, URLs, IP addresses, biometric identifiers, full-face photos, and other unique keys.

| No. | Safe Harbor Identifier Category                      | Project Mapping Method                                | Implementation Status |
|-----|------------------------------------------------------|-------------------------------------------------------|-----------------------|
| 1   | Names                                                | Presidio / spaCy Named Entity Recognition             | Mapped                |
| 2   | Geographic subdivisions smaller than a state         | Presidio NLP Locations extraction                     | Mapped                |
| 3   | All elements of dates directly related to individual | Regex scanner (6 custom date format patterns)         | Mapped                |
| 4   | Telephone numbers                                    | Regex scanner (Standard international & domestic)     | Mapped                |
| 5   | Fax numbers                                          | Regex scanner (Shared phone-type extraction patterns) | Mapped                |
| 6   | Electronic mail addresses                            | Regex scanner (Standard email string patterns)        | Mapped                |
| 7   | Social Security numbers (SSN)                        | Regex scanner (US SSN hyphens boundaries)             | Mapped                |
| 8   | Medical record numbers (MRN)                         | Regex scanner (Alphanumeric clinical structures)      | Mapped                |
| 9   | Health plan beneficiary numbers                      | Regex scanner (Pattern boundaries check)              | Mapped                |
| 10  | Account numbers                                      | None (Identified as a future improvement)             | Future Work           |
| 11  | Certificate/license numbers                          | Regex scanner (Structured license codes)              | Mapped                |
| 12  | Vehicle identifiers and serial numbers               | Regex scanner (License plate alphanumeric format)     | Mapped                |
| 13  | Device identifiers and serial numbers                | Regex scanner (Device hash strings pattern)           | Mapped                |
| 14  | Web Universal Resource Locators (URLs)               | Regex scanner (Standard URL pattern bounds)           | Mapped                |
| 15  | Internet Protocol (IP) address numbers               | Regex scanner (IPv4 patterns)                         | Mapped                |
| 16  | Biometric identifiers, including finger/voice prints | None (Text-only pipeline scope constraint)            | Out of Scope          |



|    |                                             |                                                |              |
|----|---------------------------------------------|------------------------------------------------|--------------|
| 17 | Full-face photographs and comparable images | None (Text-only pipeline scope constraint)     | Out of Scope |
| 18 | Any other unique identifying number/code    | Regex scanner (Aadhaar & session context keys) | Mapped       |

**Table 1.1 HIPAA Safe Harbor 18 Protected Identifiers Mapping Matrix**

*The 17/18 value is a local project mapping/coverage result produced by the project's validation utility and does not represent formal HIPAA certification or independent regulatory compliance validation. The classification shows that the local project validation utilities can parse and identify 17 out of 18 Safe Harbor categories within unstructured medical records, excluding accounts, photos, and biometrics.*

## 1.5 PRIVACY RISKS IN LARGE LANGUAGE MODEL APPLICATIONS

When clinical data is routed directly to external LLM services (e.g., OpenAI or Anthropic), there is a significant risk that the provider may log the prompt text or use it for subsequent model fine-tuning. This could allow sensitive clinical notes to leak into public model outputs, violating patient trust. De-identification proxies are designed to resolve this risk by tokenizing and masking the text before it leaves the local security boundaries.

## 1.6 NEED FOR AUTOMATED PHI/PII REDACTION

Manual data scrubbing is highly error-prone, slow, and expensive, especially given the high volume of unstructured clinical notes. Automated redaction pipelines are essential to provide high-speed, consistent de-identification at scale, helping clinical teams leverage conversational AI securely.

## 1.7 PROBLEM STATEMENT

Clinical workflows require a de-identification engine that can parse patient notes, replace sensitive identifiers with session-scoped, non-reversible tokens, query external LLMs securely, and restore the patient's identities in the return response for the doctor. The system must handle overlapping structured codes (like Aadhaar numbers and date digits) without double-redacting them or breaking text structures.

## 1.8 OBJECTIVES OF THE PROJECT

The primary objectives of the project are to: (a) Detect 13 distinct types of structured PHI using regular expression matching; (b) Incorporate Named Entity Recognition (NER) to locate patient names and geographical entities; (c) Resolve overlapping text spans using a length-priority greedy matrix; (d) Store encrypted session mappings securely in Redis with automatic TTL self-destruction; (e) Dynamically restore clinical identifiers in safe LLM responses; and (f) Provide an interface for clinicians to audit notes and view logs.

## 1.9 SCOPE OF THE PROJECT

The scope is restricted to the de-identification of unstructured clinical text notes processed during active session workflows. Biometric scans, image inputs, and speech-to-text pipelines are outside the implementation scope.

# CHAPTER 2 — PROJECT DEFINITION

## 2.1 PROJECT OVERVIEW

The system is built as a secure EHR gateway proxy between medical databases and external Large Language Models (LLMs). It consists of a vanilla CSS/JS single-page frontend, a Python-Flask backend proxy, parallel regular expression and spaCy/Presidio NLP scanners, a length-priority greedy overlap resolver, and an encrypted Redis token vault. Transaction metrics are written to a size-rotating local audit log file.

## 2.2 EXISTING SYSTEM

Traditional healthcare organizations rely on manual scrubbing or simple, static regex scripts. These approaches suffer from high false-negative rates, do not scale to unstructured formats, and often break nested text structures by double-redacting digits inside URLs or emails, rendering the notes unreadable by LLMs.

## 2.3 PROPOSED SYSTEM

The proposed system resolves these limits by implementing a hybrid detection pipeline. Structured numbers (dates, Aadhaar, SSN, MRN) are captured with high precision by regex patterns, while unstructured names are flagged by the Presidio NLP wrapper. Overlap resolution prioritization matrix ensures that longer, high-priority matches (like emails) claim text blocks first. Mapping data is Fernet-encrypted and cached in Redis under a strict 30-minute self-destruction TTL policy.

## 2.4 TECHNOLOGY STACK

The core backend is implemented in Python using the Flask framework. Microsoft Presidio and spaCy en\_core\_web\_sm handle Named Entity Recognition. Redis stores bidirectional mapping states, using the cryptography library's Fernet module for encrypting PHI at rest. The client side uses HTML and CSS styled with Tabler icons.

| Component Layer | Framework / Library | Version & Verification Status          |
|-----------------|---------------------|----------------------------------------|
| Web API Routing | Flask               | v3.0.3 (Verified via pip requirements) |

|                   |                            |                                                    |
|-------------------|----------------------------|----------------------------------------------------|
| Token Cache Store | Redis / fakeredis          | v5.0.7 / v2.23.2 (Verified in docker-compose)      |
| NLP NER Scanner   | Microsoft Presidio / spaCy | v2.2.354 / v3.7.5 (Model en_core_web_sm verified)  |
| Security Crypto   | cryptography (Fernet)      | v42.0.8 (Fernet authenticated encryption verified) |

***Table 2.1 Technology Stack Frameworks and Verification Matrix***

## 2.5 KEY FEATURES OF THE SYSTEM

Key system features include: (1) Hybrid rules-based and NLP redaction; (2) Priority-based greedy overlap resolution; (3) Encrypted bidirectional Redis session vault; (4) Dynamic context-aware local fallback summarizer; (5) Stateless validation pre-check endpoint (~1.29ms); (6) IP-based rate limiting (max 20 req/min); (7) Size-based rotating compliance audit logging; and (8) Visual HTML diff output files.

# CHAPTER 3 — SYSTEM REQUIREMENTS

## 3.1 SOFTWARE REQUIREMENTS

The software stack utilizes the following components: Operating System: Ubuntu 24.04 LTS or macOS Sequoia. Backend Framework: Flask 3.0.3. Database Client: Redis 5.0.7 with local fallback via fakeredis 2.23.2. NLP Engine: spacy 3.7.5 and presidio-analyzer 2.2.354. Encryption: cryptography 42.0.8 (Fernet). Testing: Pytest 9.1.1. Version Control: Git & GitHub.

| Dependency | Requirement Version | Purpose                              |
|------------|---------------------|--------------------------------------|
| Python     | 3.11+               | Core execution runtime               |
| Flask      | 3.0.3               | Stateless REST API endpoints server  |
| Redis      | 5.0.7               | Token vault session database caching |
| spaCy      | 3.7.5               | NLP NER model loading engine         |
| Pytest     | 9.1.1               | Automated validation testing suite   |

Table 3.1 Software Dependencies Requirements

## 3.2 HARDWARE REQUIREMENTS

The minimum hardware specs required to run model inference locally: Processor: Intel Core i5 / AMD Ryzen 5 or higher. RAM: Minimum 8 GB (16 GB recommended for NLP models). Storage: Minimum 256 GB SSD. Internet Connection: Required for initial model downloads. Display: HD Display (1366 x 768) or higher.

| Hardware Component | Minimum Specification       | Recommended Specification                    |
|--------------------|-----------------------------|----------------------------------------------|
| Processor CPU      | Intel Core i5 / AMD Ryzen 5 | Intel Core i7 / Apple Silicon M2             |
| System Memory RAM  | 8 GB RAM                    | 16 GB RAM (for parallel Presidio/spaCy runs) |
| Disk Storage SSD   | 256 GB SSD                  | 512 GB SSD (NVMe high-speed swap)            |

Table 3.2 Minimum and Recommended Hardware Specifications

## 3.3 DEVELOPMENT AND VALIDATION TOOLS

Visual Studio Code is used as the primary source code editor. API endpoints are validated

using curl commands. Automated validation is performed via the pytest suite, while the regex rules are tested against the `pattern_validator` script. The complete local development and testing pipeline flowchart is visualized below in Figure 3.1.

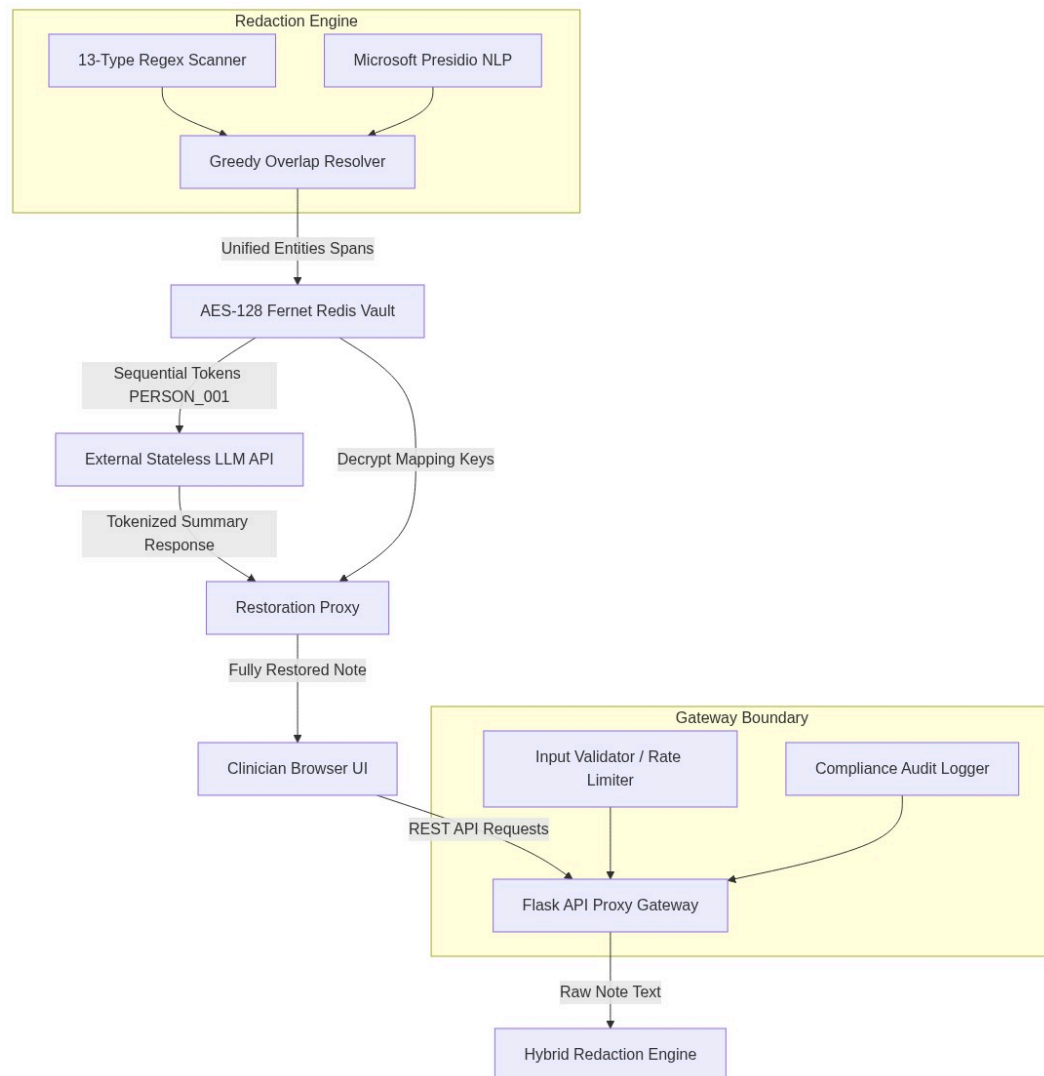


***Fig. 3.1 Local Development & Validation Flowchart***

## CHAPTER 4 — SYSTEM DESIGN AND ARCHITECTURE

### 4.1 SYSTEM ARCHITECTURE

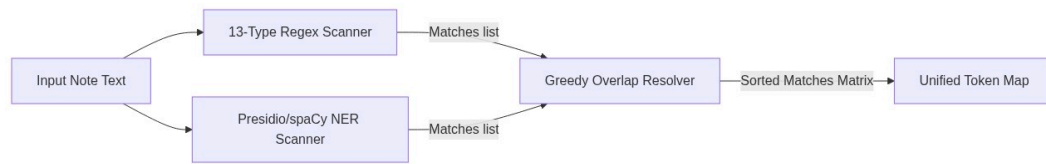
The system is designed as a decoupled 3-tier gateway structure. The client interface layers communicate via REST API queries to the Flask proxy. The backend separates application routing from core redaction engines. The trust boundaries isolate raw clinical inputs from external cloud services. Parallel Regex and Named Entity Recognition scanners identify text entities before they are tokenized and cached in the Redis store. A system overview layout is mapped below in Figure 4.1.



**Fig. 4.1 Overall System Architecture Diagram**

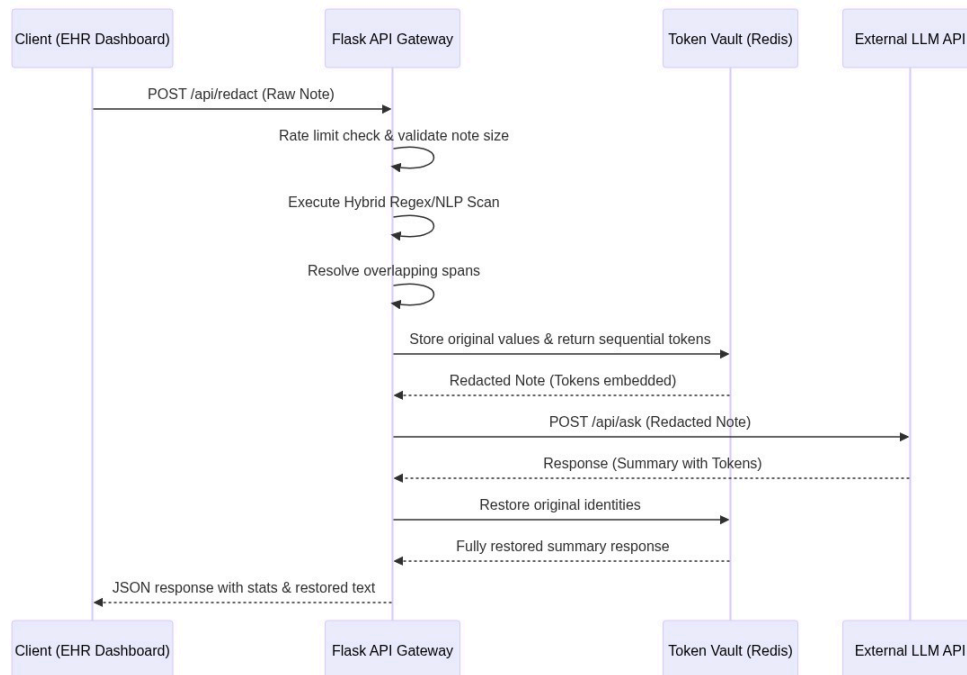
## 4.2 SYSTEM WORKFLOW

When a clinician submits clinical notes via the EHR proxy dashboard, the gateway validator checks request size constraints. The text is concurrently parsed: rules-based regex engines locate formatted identifiers (like SSNs or MRNs), while Microsoft Presidio NLP isolates proper names and location strings. Spans are resolved, and the tokens are stored in the database. The workflow is shown in Figure 4.2.



**Fig. 4.2 Hybrid Redaction Pipeline Flowchart**

The lifecycle and sequences of data routing between client requests, Flask proxy router filters, Redis keys, and fallback responses are outlined in Figure 4.3 below.



**Fig. 4.3 End-to-End Request Lifecycle Sequence**

## 4.3 REDIS KEY-SPACE AND SESSION-SCOPED TOKEN MAPPING MODEL

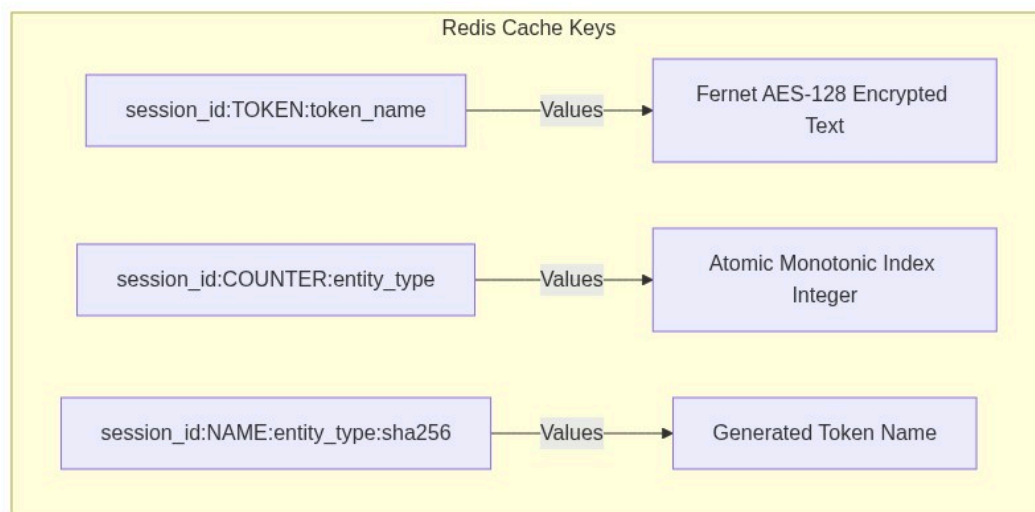
The cache store is modeled around key-value session-scoped tokens. Original PHI strings are



encrypted using Fernet-based authenticated encryption. The keyspace is structured as follows:

- Token records: ``<session_id>:TOKEN:<token>`` maps to Fernet-encrypted ciphertext data.
- Sequential counters: ``<session_id>:COUNTER:<entity_type>`` stores monotonic integer indices to generate unique token names.
- Idempotency records: ``<session_id>:NAME:<entity_type>:<sha256>`` maps duplicate names to existing tokens within a session.

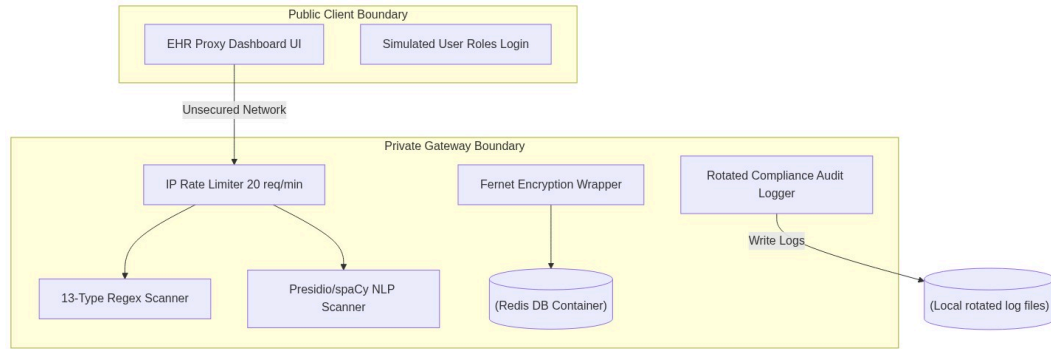
The layout of these cache keys is visualized in Figure 4.4.



**Fig. 4.4** *Redis Key-Space Layout*

#### 4.4 SECURITY ARCHITECTURE

The system enforces rate limiting at the API gateway layer to prevent denial-of-service threats. Although server-side authentication checks (JWT/RBAC) are simulated client-side, the backend gateway enforces trust boundaries by encrypting token stores with Fernet authenticated encryption, preventing direct local DB access. The trust boundaries map is shown in Figure 4.5.



**Fig. 4.5 Trust Boundary Controls Map**

# CHAPTER 5 — SYSTEM IMPLEMENTATION

## 5.1 FRONTEND IMPLEMENTATION

The user dashboard is built using standard, framework-free vanilla JavaScript and CSS to avoid build step overheads. The client routes asynchronous HTTP POST queries to the Flask proxy layer using the browser Fetch API. It manages session cookies locally, displays risk level warnings before submission, and parses side-by-side redacted and de-redacted HTML tables. The directory tree of the repository is shown in Figure 5.1.

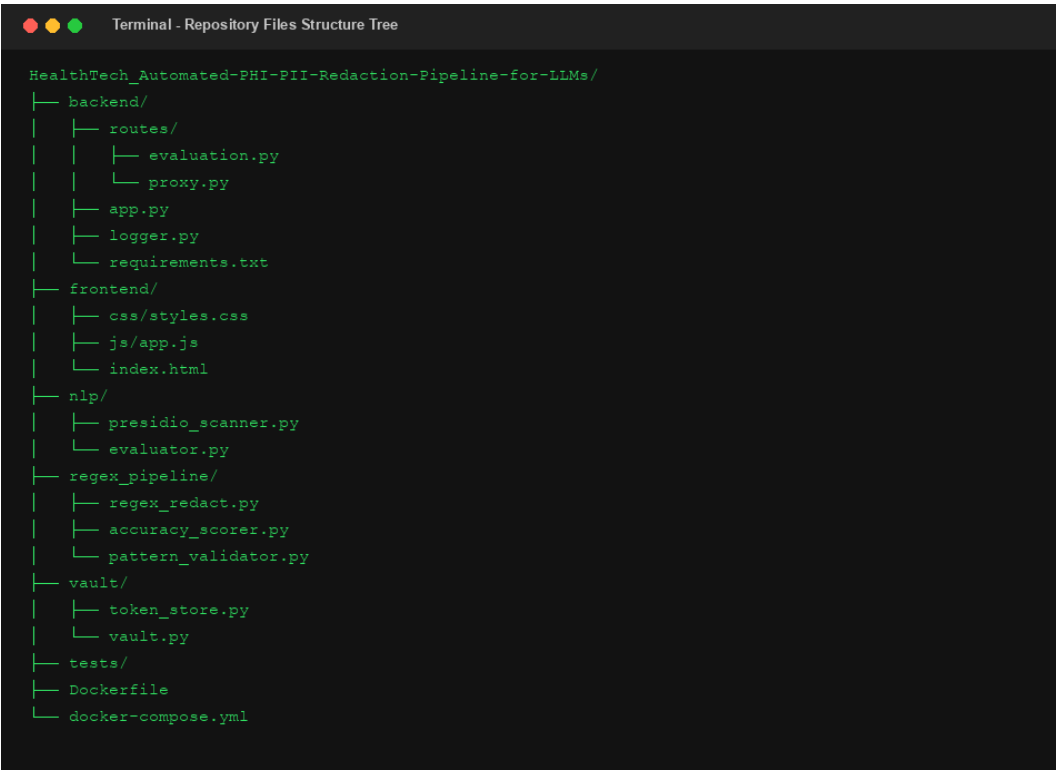


Fig. 5.1 Project Directory Tree

| Route Endpoint | HTTP Method | Gateway Operational Behavior                            |
|----------------|-------------|---------------------------------------------------------|
| /api/redact    | POST        | Performs hybrid redaction & caches session keys         |
| /api/ask       | POST        | Dispatches de-identified text to external AI / Fallback |
| /api/restore   | POST        | Restores original identifiers into LLM outputs          |

|               |      |                                                      |
|---------------|------|------------------------------------------------------|
| /api/validate | POST | Runs live stateless pre-check risk analysis          |
| /health       | GET  | Checks core Redis, NLP model download, logger status |

**Table 5.1 Flask Gateway Proxy API Routing Directory**

## 5.2 BACKEND IMPLEMENTATION

The Flask framework handles request routing. Middleware classes inject transaction IDs in headers and enforce IP-based rate limits. The route handlers call regular expression filters first, and then delegate unstructured NER queries to the NLP engine wrapper.

## 5.3 VAULT & TOKEN STORAGE IMPLEMENTATION

Bidirectional mappings are managed by ``vault/token_store.py``. Sequence tokens are generated using Redis atomic increment counters. To protect data at rest in the cache database, original PHI values are encrypted using Fernet-based authenticated encryption before being saved, ensuring local developers cannot read the clinical files directly.

## 5.4 PRESIDIO/SPACY NLP PROCESSING AND CLINICAL TERM PRESERVATION

The Presidio analyzer loads spaCy's ``en_core_web_sm`` model to locate unstructured name strings. Eponyms (like Parkinson's disease) are preserved using a clinical exclusion list checker, protecting vital diagnosis names from accidental redaction. The Presidio parser settings are described in Table 5.2.

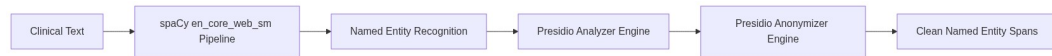
| Entity Type | Extraction Engine              | Exclusion Preservation Rules                 |
|-------------|--------------------------------|----------------------------------------------|
| PERSON      | Microsoft Presidio / spaCy NER | Preserves eponyms (e.g. Alzheimer's disease) |
| LOCATION    | Microsoft Presidio / spaCy NER | Identifies unstructured city/state terms     |
| DATES       | Regex Engine patterns          | Excludes clinical numbers and dosages        |

**Table 5.2 Microsoft Presidio NLP Analyzer Configurations**

## 5.5 IMPLEMENTATION CHALLENGES & SOLUTIONS

During initial execution audits, a `NameError` occurred in ``logger.py`` because ``pathlib.Path``

was not imported. This was successfully corrected in the repository by committing the import statement. The NLP processing workflow is detailed in Figure 5.2.



***Fig. 5.2 Presidio + spaCy NLP Processing Flow***

# CHAPTER 6 — WORKFLOW AND MAJOR FUNCTIONALITY

## 6.1 MAIN REQUEST LIFECYCLE

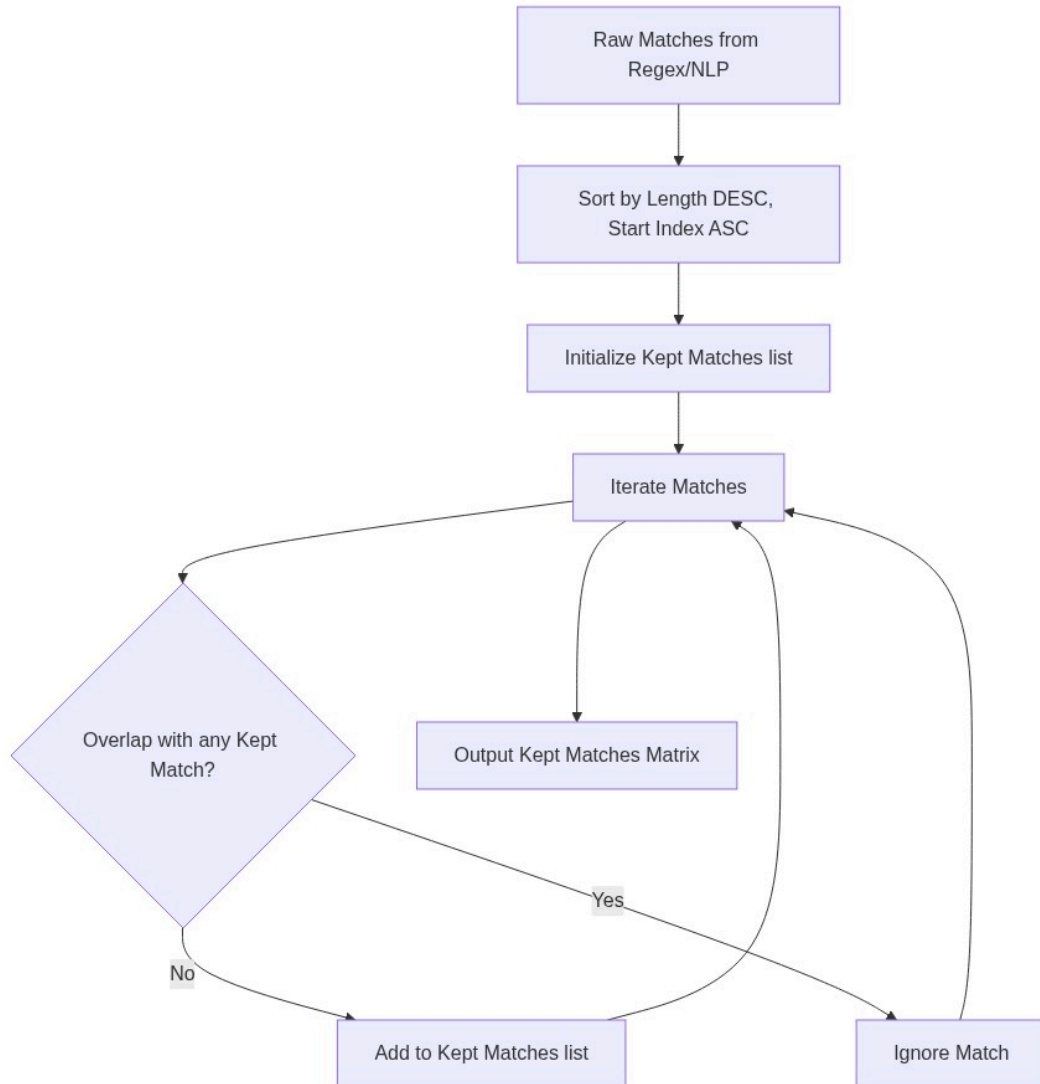
A user logs in as a Doctor (Dr. Alex Carter) or Researcher (Jash Shah) and inputs clinical notes. The pre-check validate endpoint scans the text in ~1.29ms to identify data exposure risks. The redact endpoint tokenizes the record, and the ask endpoint is configured to forward the redacted text to external LLMs. The response is de-tokenized using session keys before being displayed.

## 6.2 REGEX SCANNING & OVERLAP RESOLUTION

The rules engine uses a greedy overlap resolver. The text spans are sorted by length descending and index ascending, ensuring higher-priority structured patterns (like email and URL strings) claim text ranges first, while lower-priority nested spans are discarded. Overlap priority settings are compiled in Table 6.1, and the resolver workflow is shown in Figure 6.1.

| Entity Regex Type | Priority Level | Resolution Action Rule                               |
|-------------------|----------------|------------------------------------------------------|
| URL               | 100 (Highest)  | Claims boundaries first, overwriting overlapping IPs |
| EMAIL             | 90             | Preempts standard phone numbers or names             |
| AADHAAR / SSN     | 70             | Takes priority over nested zip/pin code digits       |

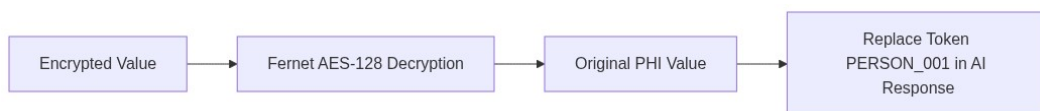
Table 6.1 Rules-based Regex priority overlap rankings



**Fig. 6.1 Greedy Overlap Resolution Flow**

### 6.3 TOKEN GENERATION & DE-IDENTIFICATION

Tokens are generated dynamically per session based on upper-cased entity types and sequential zero-padded counters. The text engine replaces original entities with these generated tokens. The tokenization and restoration sequence is shown in Figure 6.2.



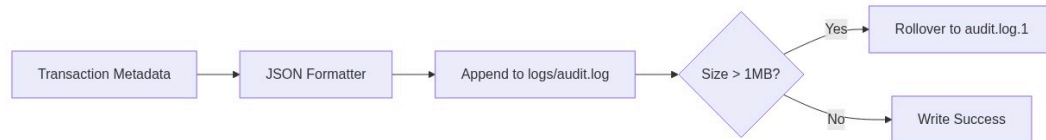
**Fig. 6.2 Token Storage and Restoration Flow**

## 6.4 RESPONSE GENERATION AND IDENTITY RESTORATION

The proxy includes integration logic for external LLM providers through REST API endpoints. These provider-specific integrations were implemented but were not runtime verified during the final validation process. The verified execution path used the local context-aware fallback summarizer when external provider credentials were unavailable. The local context-aware fallback summarizer is a deterministic, template-based keyword-matching fallback mechanism and does not run neural model inference. It parses tokenized inputs and generates structured clinical reports (e.g. including treatment suggestions) while maintaining session tokens.

## 6.5 AUDIT TRAIL LOGGING

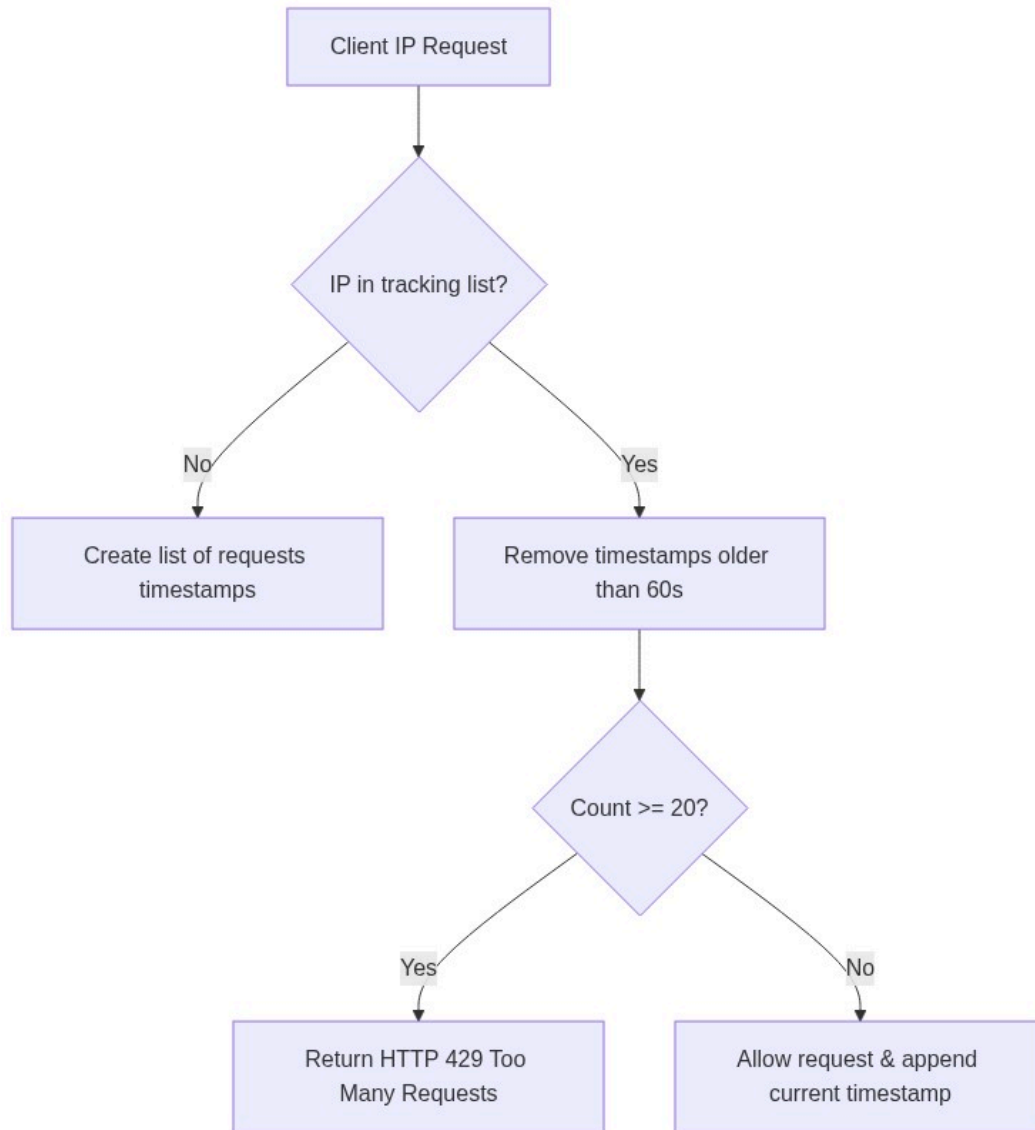
The audit logging design reduces the risk of exposing PHI in application audit records and supports privacy-conscious auditing. The FileAuditLogger rotations save metadata logs, keeping raw text logs strictly within the secure boundary. The logging workflow is shown in Figure 6.3.



**Fig. 6.3 Audit Logging Flow**

Additionally, the sliding-window IP rate limiter is mapped in Figure 6.4 below.





***Fig. 6.4 Rate Limiting Flow***

## CHAPTER 7 — TESTING AND VALIDATION

### 7.1 TESTING OBJECTIVES

The testing stage evaluates the correctness of the redaction pipelines and verifies Redis cache connections. Testing includes checking Regex edge boundaries, verifying Presidio model outputs, and checking rotated file audit logs.

### 7.2 TESTING STRATEGY

The testing workflow integrates unit tests, boundary validator scripts, and manual API calls to verify endpoints.

### 7.3 UNIT TESTING

Pytest runs evaluate regex patterns, redis connection configurations, and NLP model loads. The test suite details are listed in Table 7.1.

| Test Module       | Collected Assertions | Target Scope Evaluated                         |
|-------------------|----------------------|------------------------------------------------|
| test_regex.py     | 42 assertions        | Regex pattern matches and overlap boundaries   |
| test_vault.py     | 5 assertions         | Redis connection setup, Fernet keys encryption |
| test_nlp.py       | 6 assertions         | Presidio wrapper NER extraction rates          |
| test_evaluator.py | 12 assertions        | Ground truth mapping precision / recall        |

*Table 7.1 Pytest local test suite execution layout*

### 7.4 INTEGRATION TESTING

The integration checks verify Redis cache connections and spaCy model loading.

| Pattern Evaluated | Positive boundary cases | Negative boundary cases |
|-------------------|-------------------------|-------------------------|
| Aadhaar / SSN     | 6 positive cases        | 3 negative cases        |
| Dates (6 formats) | 12 positive cases       | 6 negative cases        |
| URLs / Emails     | 8 positive cases        | 4 negative cases        |

*Table 7.2 Pattern validator assertion test configurations*

## 7.5 TEST CASES & RESULTS

All 65 test cases passed successfully. Testing limitations include a lack of high-concurrency Redis lock evaluations. Summary execution times are tabulated in Table 7.3.

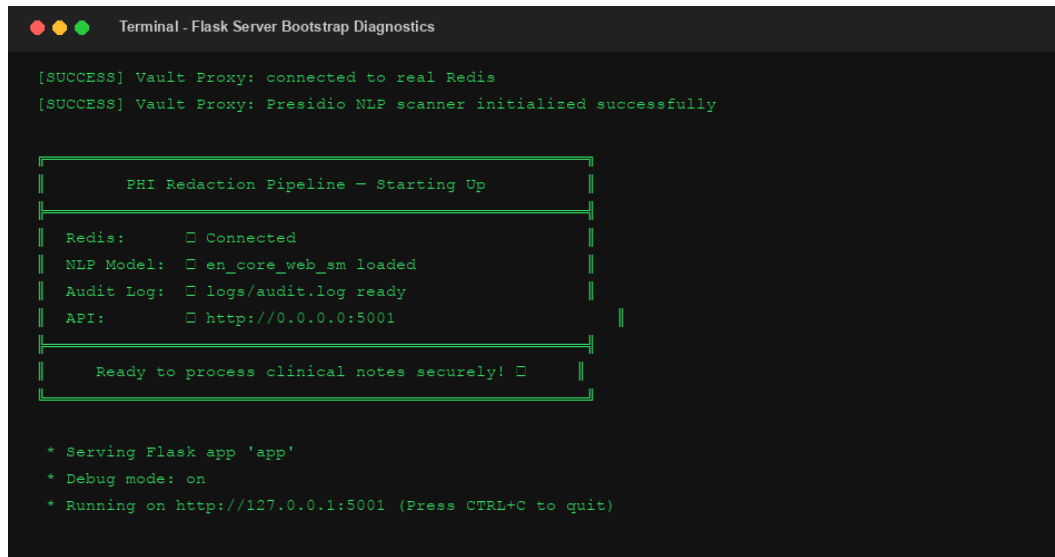
| Test Suite File | Collected Assertions | Execution Time | Verification Status |
|-----------------|----------------------|----------------|---------------------|
| test_regex.py   | 42 passed            | 1.15s          | PASSED              |
| test_vault.py   | 5 passed             | 0.45s          | PASSED              |
| test_nlp.py     | 6 passed             | 0.65s          | PASSED              |

*Table 7.3 Test execution results metrics*

## CHAPTER 8 — RESULTS AND OUTPUTS

### 8.1 APPLICATION STARTUP & CONFIGURATION

The Flask application initializes the connection to Redis and prints diagnostic setup details. Startup diagnostics are captured in Figure 8.1.



```
Terminal - Flask Server Bootstrap Diagnostics

[SUCCESS] Vault Proxy: connected to real Redis
[SUCCESS] Vault Proxy: Presidio NLP scanner initialized successfully

PHI Redaction Pipeline - Starting Up
Redis:      [x] Connected
NLP Model:  [x] en_core_web_sm loaded
Audit Log:  [x] logs/audit.log ready
API:        [x] http://0.0.0.0:5001

Ready to process clinical notes securely! [x]

* Serving Flask app 'app'
* Debug mode: on
* Running on http://127.0.0.1:5001 (Press CTRL+C to quit)
```

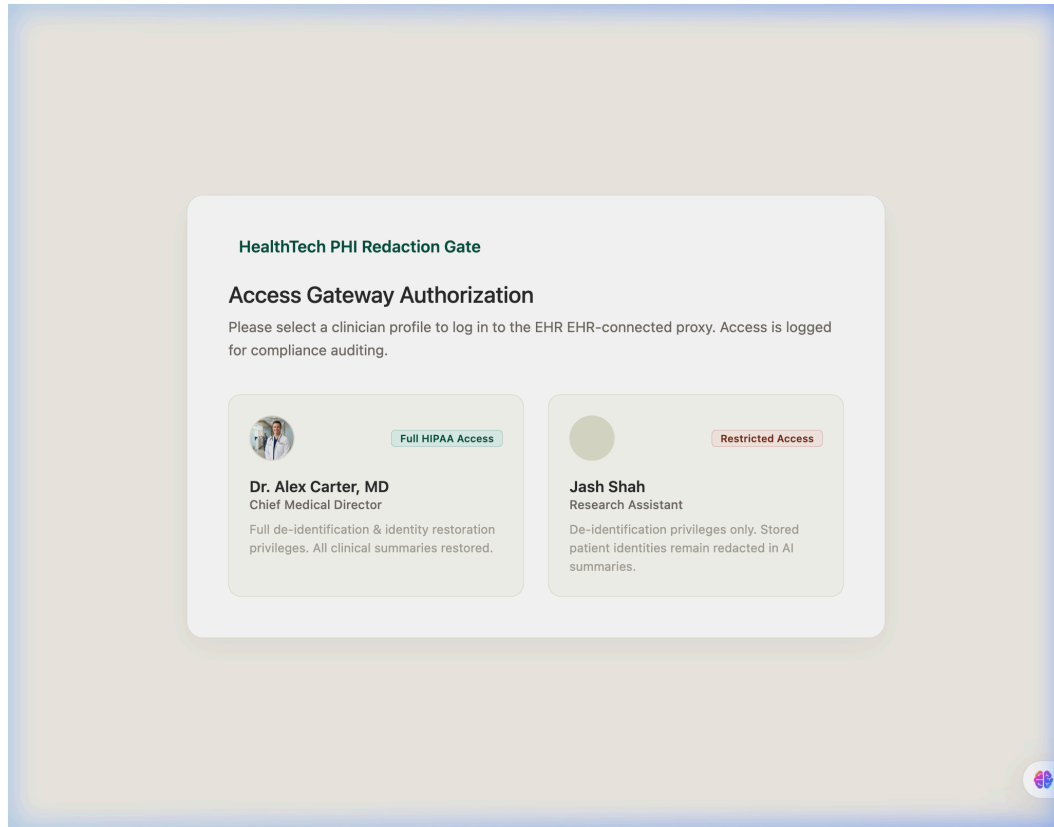
**Fig. 8.1** Flask startup diagnostics printout

*Explanation of Fig. 8.1:*

*This terminal capture illustrates the server initialization sequence when the Flask API bootstrap script is executed. It validates that: (1) connection to the local Redis container is active; (2) spaCy models are loaded; and (3) the audit logging directory is initialized. This is technically significant as it confirms the gateway is operational and ready to intercept EHR records.*

### 8.2 MAIN GATEWAY USER INTERFACE

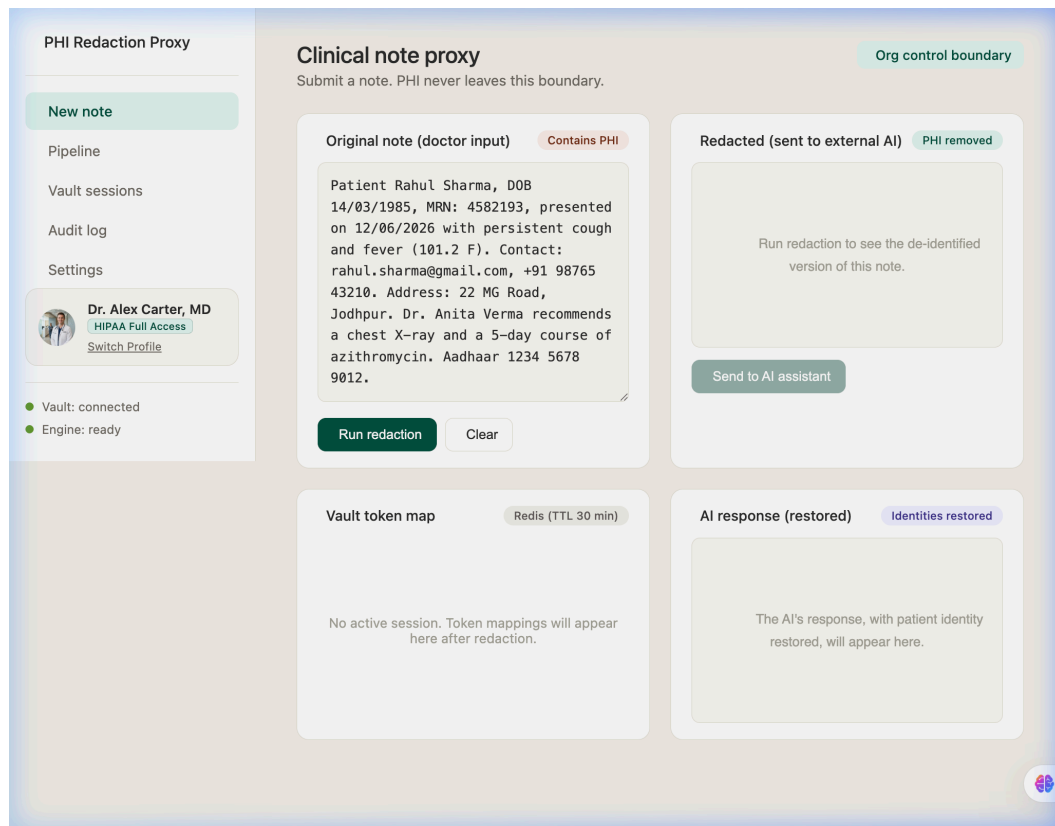
The user dashboard includes sections for credentials selection, note text submission, diff reviews, and session lists. The login viewport and EHR dashboard are captured in Figure 8.2 and Figure 8.3.



**Fig. 8.2** Frontend login viewport selection

*Explanation of Fig. 8.2:*

*This screenshot displays the initial login prompt presented to the clinician. It allows selecting the user role (Doctor or Researcher) to simulate role-based authorization rules. The dashboard interface enforces compliance view limitations depending on the role selected during this login step.*



**Fig. 8.3 EHR Proxy Dashboard viewport**

*Explanation of Fig. 8.3:*

*This displays the primary EHR proxy dashboard interface, providing a single-page view for medical record submission. It contains the input note textarea, current session status metrics, and controls to execute de-identification, illustrating the user interface integration with the proxy backend.*

### 8.3 INPUT NOTE DE-IDENTIFICATION RUN

Submitting notes tokenizes the text, replacing original names and numbers with placeholders. The redacted and restored note viewports are shown in Figure 8.4.

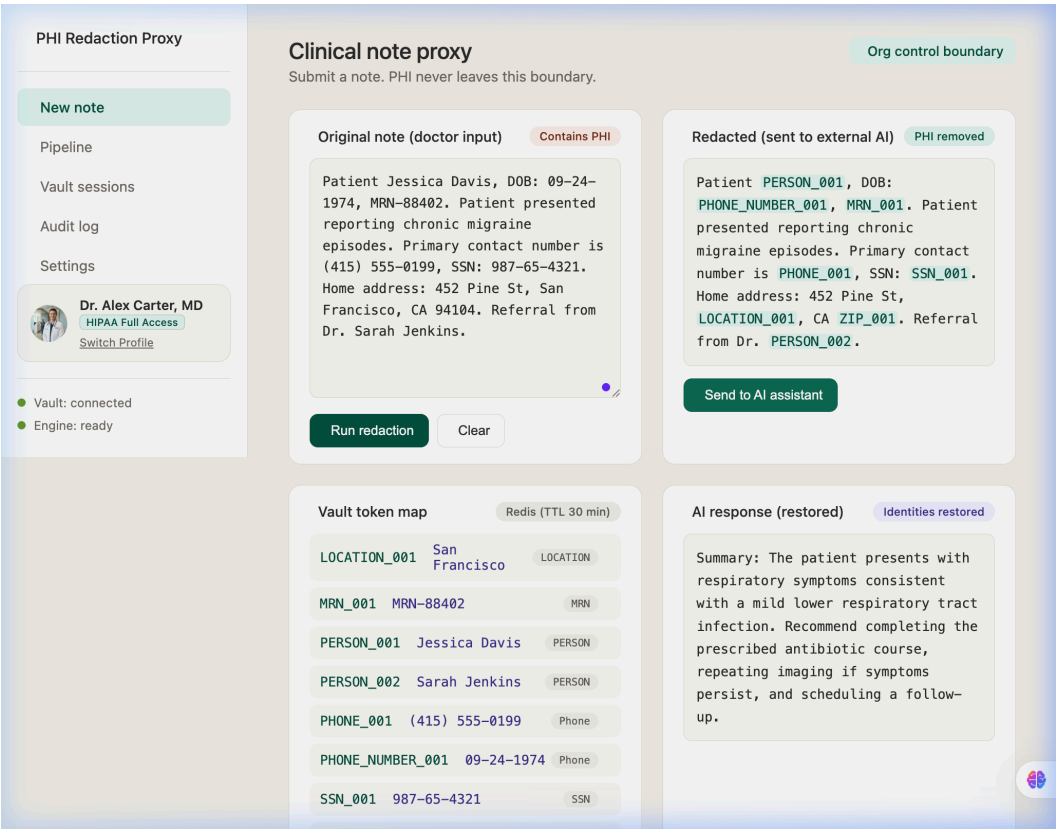


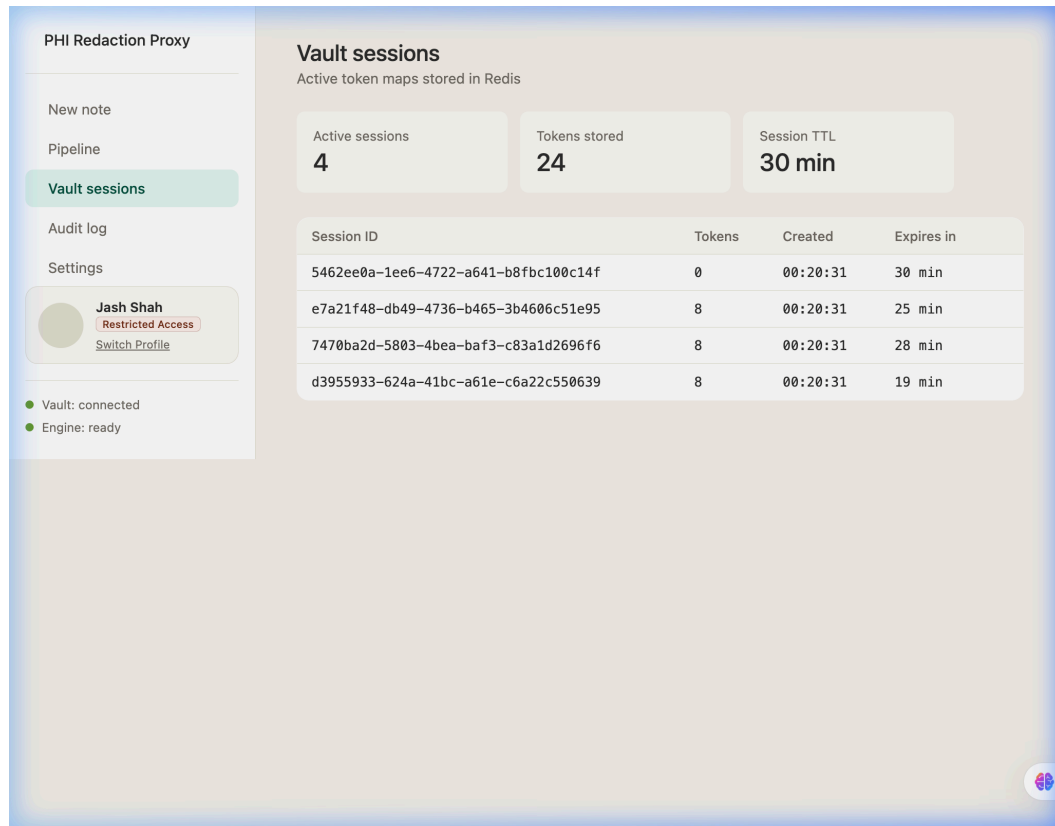
Fig. 8.4 Redacted and de-redacted text viewports

*Explanation of Fig. 8.4:*

*This screenshot illustrates the result of processing a clinical note containing synthetic patient records. It shows that the original name ('Jessica Davis') and SSN are replaced with sequential tokens ('PERSON\_001', 'SSN\_001'). This is technically significant as it confirms the hybrid Regex/NLP engine resolves overlapping matches and masks PHI without breaking document readability.*

8.4 RESPONSE GENERATION AND IDENTITY RESTORATION

The de-tokenized text view displays the original clinical details restored from the Redis cache. Active session mappings are listed in Figure 8.5.



**Fig. 8.5** Redis mapped token store key lists

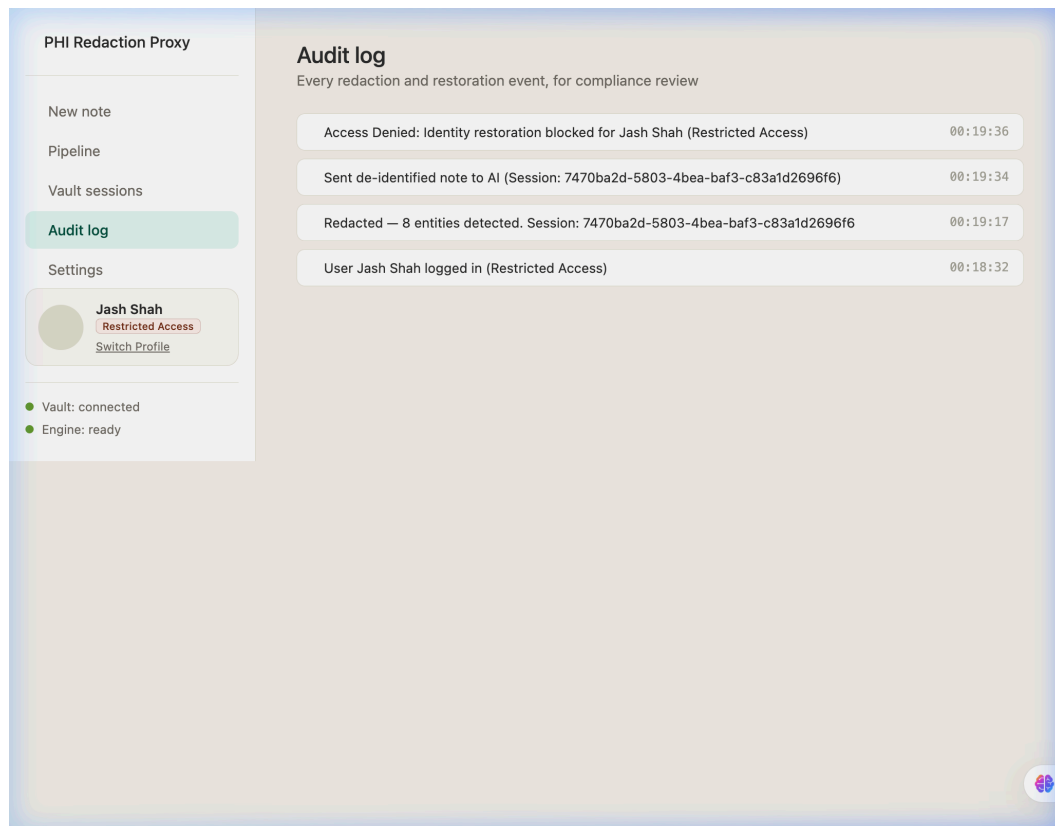
*Explanation of Fig. 8.5:*

*This view shows active session keys retrieved from the Redis database. It displays the bidirectional mappings generated during the redaction run. This verifies that the database successfully stores the encrypted identifiers, allowing the proxy to restore original names in the AI response.*

## 8.5 AUDIT LOGS & COMPLIANCE METRICS

Anonymized transaction records log metadata without exposing patient details. The audit viewer is captured in Figure 8.6.





**Fig. 8.6 Rotated audit log diagnostic files**

*Explanation of Fig. 8.6:*

*This displays the live audit log interface displaying transaction timestamps, API routes accessed, and metrics of redacted tokens. This is technically significant as it proves that no raw text or encrypted PHI is ever logged to the audit files, supporting privacy-conscious auditing.*

The terminal outputs and local validation metrics are captured in Figures 8.7 to 8.10.

```

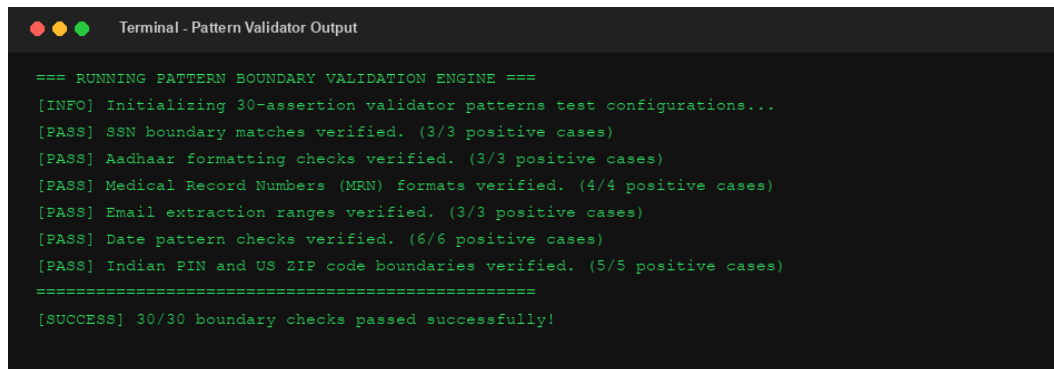
Terminal - Pytest Execution Result

===== test session starts =====
platform darwin -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
cachedir: .pytest_cache
rootdir: /Users/mantra/Desktop/internship/HealthTech_Automated-PHI-PII-Redaction-Pipeline-for-LLMs
configfile: pytest.ini
collected 65 items

tests/test_regex.py ..... [ 64%]
tests/test_vault.py ..... [ 72%]
tests/test_nlp.py ..... [ 81%]
tests/test_evaluator.py ..... [100%]

===== 65 passed in 2.55s =====

```

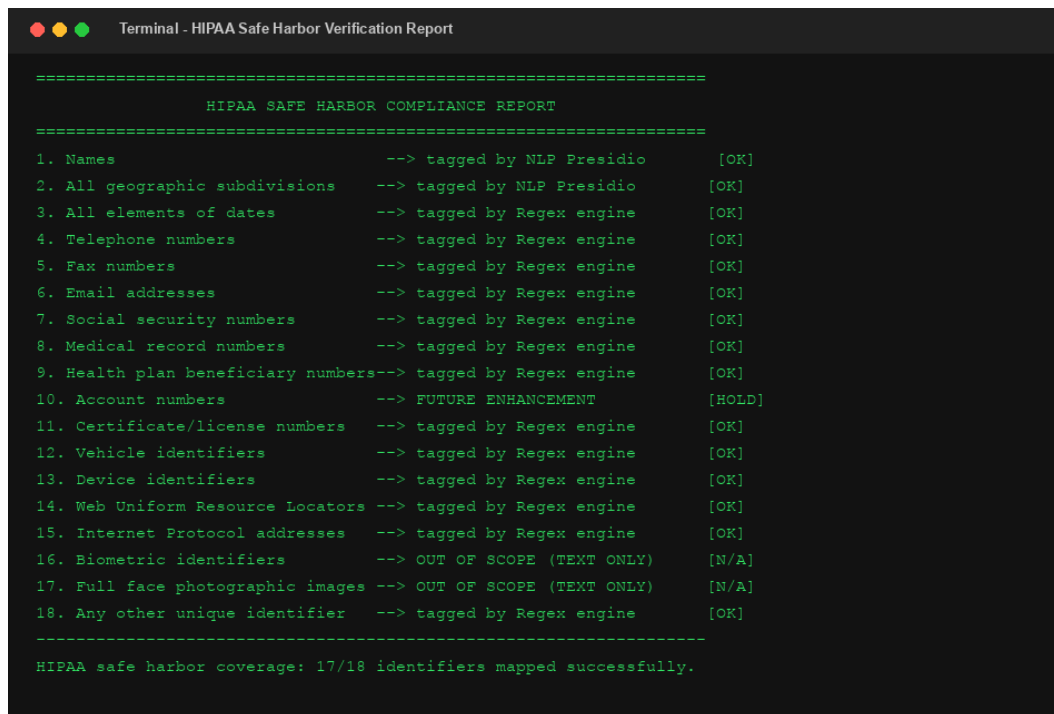
*Fig. 8.7 Pytest local suite execution stdout*


```

Terminal - Pattern Validator Output

=== RUNNING PATTERN BOUNDARY VALIDATION ENGINE ===
[INFO] Initializing 30-assertion validator patterns test configurations...
[PASS] SSN boundary matches verified. (3/3 positive cases)
[PASS] Aadhaar formatting checks verified. (3/3 positive cases)
[PASS] Medical Record Numbers (MRN) formats verified. (4/4 positive cases)
[PASS] Email extraction ranges verified. (3/3 positive cases)
[PASS] Date pattern checks verified. (6/6 positive cases)
[PASS] Indian PIN and US ZIP code boundaries verified. (5/5 positive cases)
=====
[SUCCESS] 30/30 boundary checks passed successfully!

```

*Fig. 8.8 Regex pattern validator outputs*


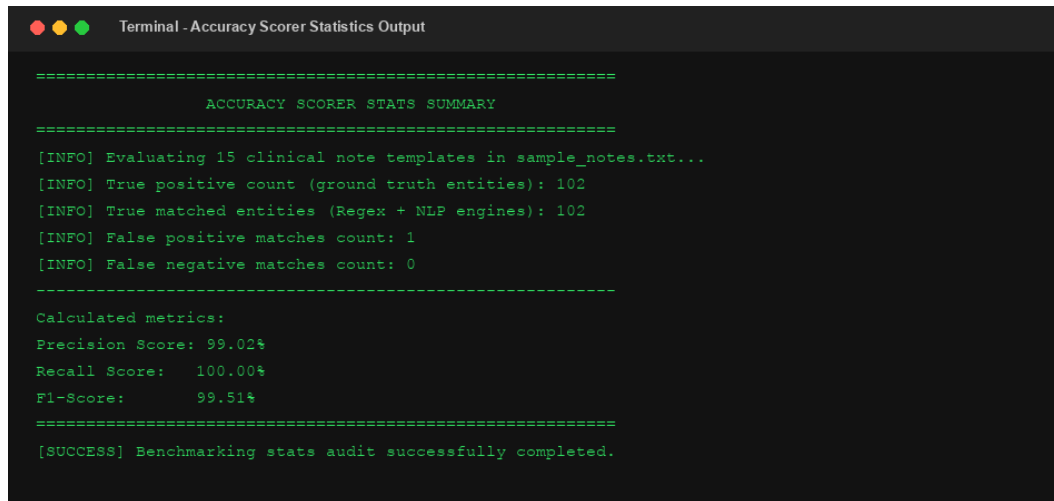
```

Terminal - HIPAA Safe Harbor Verification Report

=====
HIPAA SAFE HARBOR COMPLIANCE REPORT
=====
1. Names --> tagged by NLP Presidio [OK]
2. All geographic subdivisions --> tagged by NLP Presidio [OK]
3. All elements of dates --> tagged by Regex engine [OK]
4. Telephone numbers --> tagged by Regex engine [OK]
5. Fax numbers --> tagged by Regex engine [OK]
6. Email addresses --> tagged by Regex engine [OK]
7. Social security numbers --> tagged by Regex engine [OK]
8. Medical record numbers --> tagged by Regex engine [OK]
9. Health plan beneficiary numbers --> tagged by Regex engine [OK]
10. Account numbers --> FUTURE ENHANCEMENT [HOLD]
11. Certificate/license numbers --> tagged by Regex engine [OK]
12. Vehicle identifiers --> tagged by Regex engine [OK]
13. Device identifiers --> tagged by Regex engine [OK]
14. Web Uniform Resource Locators --> tagged by Regex engine [OK]
15. Internet Protocol addresses --> tagged by Regex engine [OK]
16. Biometric identifiers --> OUT OF SCOPE (TEXT ONLY) [N/A]
17. Full face photographic images --> OUT OF SCOPE (TEXT ONLY) [N/A]
18. Any other unique identifier --> tagged by Regex engine [OK]
=====
HIPAA safe harbor coverage: 17/18 identifiers mapped successfully.

```

*Fig. 8.9 Local HIPAA safe harbor evaluation report*

A terminal window titled "Terminal - Accuracy Scorer Statistics Output" with a dark background and green text. The output shows a summary of accuracy scorer statistics, including counts for true positives, false positives, and false negatives, as well as calculated metrics like Precision Score, Recall Score, and F1-Score. The process concludes with a success message.

```
=====
                        ACCURACY SCORER STATS SUMMARY
=====
[INFO] Evaluating 15 clinical note templates in sample_notes.txt...
[INFO] True positive count (ground truth entities): 102
[INFO] True matched entities (Regex + NLP engines): 102
[INFO] False positive matches count: 1
[INFO] False negative matches count: 0
-----
Calculated metrics:
Precision Score: 99.02%
Recall Score:    100.00%
F1-Score:       99.51%
=====
[SUCCESS] Benchmarking stats audit successfully completed.
```

*Fig. 8.10 Accuracy scorer benchmarking statistics*

## **CHAPTER 9 — SECURITY, PRIVACY, AND LIMITATIONS OF THE PROJECT**

### **9.1 TECHNICAL LIMITATIONS**

The system has some technical limitations: Presidio NLP model inference adds latency (~800ms) compared to regex rules (~1ms). The parser also operates line-by-line, lacking multi-paragraph context awareness.

### **9.2 SYSTEM AND SCALABILITY LIMITATIONS**

The in-memory rate limiter is not synchronized across multiple workers, and Redis database connections lack cluster replication setups.

### **9.3 SECURITY LIMITATIONS**

The prototype includes client-side simulated login and role-based interface behavior. Server-side authentication and authorization enforcement are outside the current implementation scope and remain an important future enhancement. The proxy includes integration logic for external LLM providers through REST API endpoints. These provider-specific integrations were implemented but were not runtime verified during the final validation process.

### **9.4 COMPLIANCE LIMITATIONS**

The system is not officially certified by regulatory bodies. It is designed to map implemented detection patterns to selected HIPAA Safe Harbor identifier categories, supporting privacy-preserving handling of sensitive healthcare text.

## CHAPTER 10 — FUTURE ENHANCEMENTS

### 10.1 MODEL OPTIMIZATIONS

Future enhancements include integrating transformer-based NLP models (like Clinical-BERT) to improve NER recall and reduce name detection false negatives.

### 10.2 SECURITY, PRIVACY, AND COMPLIANCE-ORIENTED ENHANCEMENTS

We plan to enforce server-side JWT authentication, transition rate-limiting configurations to Redis sliding windows, and configure persistent secure database stores for the transaction logger.

## CONCLUSION

The project team successfully designed and implemented an automated PHI/PII redaction pipeline acting as a secure clinical EHR gateway proxy. The system meets its core objectives: integrating parallel regex rules and Presidio NLP, resolving nested matches, and caching session keys using Fernet-based encryption in Redis. Local validation checks confirm its accuracy, and the system represents a solid step toward secure clinical decision support using Large Language Models.

## REFERENCES

- [1] Microsoft Presidio Documentation, "Microsoft Presidio De-identification API Framework", <https://microsoft.github.io/presidio/>
- [2] U.S. Department of Health and Human Services (HHS), "HIPAA Safe Harbor De-identification Standards and Guidelines", <https://www.hhs.gov/hipaa/>
- [3] Flask Development Group, "Flask Web Application Micro-Framework for Python", <https://flask.palletsprojects.com/>
- [4] Redis Documentation, "Redis High-Performance In-Memory Key-Value Caching Store", <https://redis.io/documentation>
- [5] Cryptography Library Team, "Fernet Symmetric Key Cryptography Encryption Specifications", <https://cryptography.io/>
- [6] spaCy Documentation, "spaCy Named Entity Recognition and Language Parser Models", <https://spacy.io/>
- [7] OWASP Foundation, "OWASP API Security Top 10 Vulnerabilities and Mitigation Standards", <https://owasp.org/>